Prometheus, there. In agent mode, Prometheus deactivates several functions, including the UI, alerting, and local storage.

- *Graphite (https://grafana.com/oss/graphite/)*—Now also under the Grafana Labs umbrella, this agent represents an example of the telemetry of the previous generation of machine-centric monitoring.
- *Nagios (https://www.nagios.org/)*—This is a venerable open source system (including agent) for monitoring machines and networks.
- *StatsD (https://github.com/statsd/statsd)*—Developed by Etsy, this agent is a Node.js tool that has a client–server model, allowing you to collect and aggregate application metrics and send the metrics to backends, such as Graphite.
- *Telegraf (https://github.com/influxdata/telegraf)*—From InfluxData, this is a rich and powerful agent with many plug-ins, originally written for use with their InfluxDB time series open source datastore and now also supporting OpenTelemetry.

This section is intentionally brief, since I think it's important to be aware of other agents in this space; however, in practice, you can expect that in the majority of cases, you will be dealing with Prometheus, in one form or another. If you come across a legacy environment where one of the preceding agents is used, you want to make sure that there is a clear path to migrate to Prometheus or OpenTelemetry in its place.

4.3 OpenTelemetry

OpenTelemetry is an incubating cloud-native computing foundation project that started out as a merger of two projects, OpenCensus and OpenTracing, that were focused on distributed traces. OpenTelemetry expanded its scope, starting in 2019, to metrics and logs. The project is the second-largest CNCF project after Kubernetes, and in its core, it is a collection of APIs, SDKs, and tools you can use to instrument, collect, process, and ingest telemetry signals.

It provides a vendor-agnostic model for emitting traces, metrics, logs (and, going forward, also profiles [https://github.com/open-telemetry/oteps/issues/139]) with both backward compatibility (via receivers and exporters) and native functions via a dedicated protocol. OpenTelemetry includes auto-instrumentation support for some popular program languages and runtimes, such as Java/JVM, .NET, JavaScript/Node.js, and Python. If you're interested in the current status of the many components OpenTelemetry has, you can

- Check out the high-level project status (https://opentelemetry.io/status/) page.
- View fine-grained information, since the project keeps track of the detailed compliance of the implementations with the specification (http://mng.bz/Aojp) as well.

Note that in addition to the specifications (APIs and data model), OpenTelemetry comes with a set of semantics, such as the standard names for system/runtime metric instruments (http://mng.bz/Zqmm).